

SC102 L17: 复习与展望——详细扩充学习笔记

涵盖：C# 基础 | OOP | 数据结构与算法 | 设计模式 | 图形学基础 | Shader | 引擎架构 | 未来展望

SC102 全课程期末复习 + 进阶学习方向指南

1 课程总览 (Course Overview)

SC102 是游戏开发与计算机科学的核心入门课程，涵盖了从编程基础到引擎实战的完整知识链条。本学期共学习了两大主题方向：

- **编程与软件工程**：C# 编程基础、面向对象编程、数据结构与算法、设计模式等，为编写高质量、可维护的代码打基础。
- **图形学与引擎**：渲染管线、Shader 编程、游戏引擎架构等，让学生理解游戏从代码到画面的完整流程。

本课（L17）作为期末最后一课，将系统回顾全部知识点，并展望后续进阶学习方向（Shader 渲染、引擎工具开发、图形学进阶等），帮助学生建立从”入门”到”专精”的学习路线图。

SC102 完整知识体系：

C# 基础 → OOP → 数据结构与算法 → 设计模式 → Unity 实战 → Shader 与图形学 → 引擎理解 → 进阶专精

2 第一部分：C# 与 Unity 基础回顾

2.1 C# 编程语言基础

[p.4]

C# (C Sharp) 是由微软开发的面向对象编程语言，是 Unity 游戏引擎的主脚本语言。在 SC102 中，我们从零开始学习了 C# 的核心概念：

2.1.1 变量与类型系统 (Variables & Type System)

C# 是强类型语言 (Strongly Typed Language)，每个变量在声明时必须指定类型。核心类型包括：

- **值类型 (Value Types):** 直接存储数据, 赋值时产生副本。包括 `int` (整数)、`float` (浮点数)、`double` (双精度)、`bool` (布尔值)、`char` (字符)、`struct` (结构体)、`enum` (枚举)。
- **引用类型 (Reference Types):** 存储数据的内存地址, 赋值时共享同一对象。包括 `string` (字符串)、`class` (类)、`array` (数组)、`interface` (接口)、`delegate` (委托)。
- **值类型与引用类型的本质区别:** 值类型分配在栈 (Stack) 上, 引用类型的对象数据分配在堆 (Heap) 上。当赋值值类型时, 复制了整个数据; 当赋值引用类型时, 复制了引用 (地址), 两个变量指向同一块内存。

2.1.2 流程控制 (Flow Control)

- **条件语句:** `if-else` (分支选择)、`switch-case` (多分支匹配)。注意 C# 的 `switch` 支持模式匹配 (C# 7.0+)。
- **循环语句:** `for` (已知次数循环)、`while` (条件循环)、`do-while` (至少执行一次)、`foreach` (遍历集合)。在 Unity 中循环写 `Update()` 时需特别注意性能。
- **跳转语句:** `break` (跳出循环)、`continue` (跳过当前迭代)、`return` (返回方法)。

2.1.3 方法与函数 (Methods & Functions)

- **方法定义:** 访问修饰符返回类型方法名 (参数列表) { 方法体 }
- **参数传递:** 值类型默认按值传递 (Pass by Value); 使用 `ref` 关键字可按引用传递 (Pass by Reference); 使用 `out` 关键字用于输出多返回值。
- **方法重载 (Method Overloading):** 同一类中可以有多个同名方法, 只要参数列表 (类型或数量) 不同。这是编译时多态的一种形式。

2.2 Unity 引擎基础

[p.4]

2.2.1 Unity 核心概念

- **GameObject (游戏对象):** Unity 场景中所有物体的基类。GameObject 本身是一个空的容器, 其功能通过附加 Component 实现。每个 GameObject 都有一个 Transform 组件。
- **Component (组件):** 附加到 GameObject 上的功能模块。例如 `MeshRenderer` 负责渲染网格、`Collider` 负责碰撞检测、`Rigidbody` 负责物理模拟。
- **Transform (变换组件):** 每个 GameObject 必有且仅有一个的组件, 存储物体的位置 (Position)、旋转 (Rotation)、缩放 (Scale)。支持父子层级关系, 子物体的 Transform 相对于父物体计算。

- **Prefab (预制体)**: 将配置好的 GameObject 保存为模板, 可重复实例化。修改 Prefab 会同步更新所有实例 (除非实例有 Override)。
- **Scene (场景)**: 包含所有 GameObject 的游戏关卡/界面。场景之间可切换, 数据可通过 DontDestroyOnLoad() 跨场景保持。

2.2.2 MonoBehaviour 生命周期 (Lifecycle)

[p.4]

MonoBehaviour 是 Unity 脚本的基类。其生命周期方法在特定时机由 Unity 引擎自动调用, 执行顺序至关重要:

1. **Awake()** —— 对象创建时立即调用 (即使脚本未启用)。用于初始化自身引用。
2. **OnEnable()** —— 脚本启用时调用 (每次启用都会触发)。
3. **Start()** —— 在第一帧 Update 之前调用。用于需要其他对象已完成 Awake 时的初始化。
4. **FixedUpdate()** —— 固定时间间隔调用 (默认 0.02 秒, 与物理同步)。用于物理相关逻辑。
5. **Update()** —— 每帧调用, 频率取决于帧率。用于大部分游戏逻辑。
6. **LateUpdate()** —— Update 之后调用。用于摄像机跟随等依赖其他对象已更新的逻辑。
7. **OnDisable()** —— 脚本禁用时调用。
8. **OnDestroy()** —— 对象销毁时调用。

关键点: Awake 和 Start 都只在生命周期中调用一次 (除非被重新 Enable), 但 Awake 在 Start 之前, 且无论脚本是否启用都会执行。物理逻辑必须放在 FixedUpdate 中以保持物理模拟的一致性。

3 第二部分: 面向对象编程 (OOP) 回顾

3.1 四大核心概念

[p.5]

面向对象编程 (Object-Oriented Programming, OOP) 是一种以对象为核心的编程范式。SC102 中重点学习了 OOP 的四大基石:

3.1.1 封装 (Encapsulation)

封装将数据和操作方法绑定在一起, 隐藏内部实现细节, 仅暴露必要的接口。

- 访问修饰符:

- `public` ——任何地方可访问
 - `private` ——仅同类内部可访问（默认）
 - `protected` ——同类及子类可访问
 - `internal` ——同一程序集内可访问
- **属性 (Properties):** 通过 `get` 和 `set` 访问器控制对字段的访问，可以在读写时添加验证逻辑。
 - **封装的价值:** 防止外部代码直接修改内部状态、降低耦合度、便于未来修改内部实现而不影响外部调用方。

3.1.2 继承 (Inheritance)

继承允许一个类（子类/Derived Class）获取另一个类（父类/Base Class）的成员。

- C# 是**单继承 (Single Inheritance)** 语言，一个类只能有一个直接父类。
- `virtual` 关键字标记可被子类重写的方法；`override` 关键字在子类中重写。
- **里氏替换原则 (Liskov Substitution Principle, LSP):** 任何使用父类的地方都应该能透明地使用子类对象。
- **继承的陷阱:** 过深的继承链会增加复杂度和维护成本。现代设计倾向”组合优于继承 (Composition over Inheritance)”。

3.1.3 多态 (Polymorphism)

多态允许同一接口表现出不同的行为。

- **编译时多态 (静态多态):** 方法重载 (Method Overloading)，根据参数列表在编译时确定调用哪个版本。
- **运行时多态 (动态多态):** 方法重写 (Method Overriding)，通过 `virtual/override` 机制，在运行时根据对象实际类型决定调用哪个方法。
- **多态的核心价值:** 编写通用的、可扩展的代码。例如，可以写一个接受 `Animal` 类型的参数的方法，运行时传入 `Dog` 或 `Cat` 都会正确调用各自的 `Speak()` 方法。

3.1.4 抽象 (Abstraction)

抽象通过隐藏复杂细节，只暴露必要的功能接口。

- **抽象类 (Abstract Class):** 使用 `abstract` 关键字。不能实例化，可包含抽象方法（无实现）和具体方法（有实现）。适合”is-a”关系中的部分实现共享。

- **接口 (Interface):** 使用 `interface` 关键字。定义一组方法、属性、事件的签名，不允许有实现 (C# 8.0+ 允许默认实现)。适合“can-do”关系，允许一个类实现多个接口。
- **抽象类 vs 接口:** 抽象类用于有共同实现的继承体系；接口用于定义跨层级的能力契约。一个类只能继承一个抽象类，但可实现多个接口。

3.2 SOLID 原则简述

[p.5]

- A. **单一职责原则 (Single Responsibility Principle, SRP):** 一个类应该只有一个引起它变化的原因。每个类只负责一件事。
- B. **开闭原则 (Open-Closed Principle, OCP):** 对扩展开放，对修改关闭。新增功能应通过扩展实现，而非修改已有代码。
- C. **里氏替换原则 (Liskov Substitution, LSP):** 子类必须能够完全替代父类使用。
- D. **接口隔离原则 (Interface Segregation, ISP):** 不应强迫客户端依赖它们不使用的方法。大接口应拆分为多个小接口。
- E. **依赖倒置原则 (Dependency Inversion, DIP):** 高层模块不应依赖低层模块，两者都应依赖抽象。抽象不应依赖细节，细节应依赖抽象。

4 第三部分：数据结构与算法回顾

4.1 核心数据结构

[p.6]

4.1.1 数组与列表 (Array & List)

- **数组 (Array):** 固定长度的连续内存区域。通过索引 $O(1)$ 访问。适合已知大小的场景。声明后不可改变长度。
- **列表 (List<T>):** 动态数组，内部基于数组实现，容量不足时自动扩容（通常翻倍）。支持 `Add()`、`Remove()`、`Insert()` 等操作。
- **性能特征:** 索引访问 $O(1)$ ；末尾添加均摊 $O(1)$ ；中间插入/删除 $O(n)$ 。

4.1.2 字典 (Dictionary<TKey, TValue>)

基于哈希表 (Hash Table) 实现的键值对集合。通过键 (Key) 快速查找对应的值 (Value)，平均时间复杂度 $O(1)$ 。

- **核心原理**：通过哈希函数将键映射到哈希码，用哈希码确定存储位置（桶/Bucket）。哈希冲突通过链地址法（Separate Chaining）解决。
- **键的要求**：键类型必须正确实现了 `GetHashCode()` 和 `Equals()` 方法，且哈希码在键的整个生命周期内应保持不变。
- **HashSet<T>**：不存储值的”字典”，仅存储不重复的键。用于去重和快速查找。

4.1.3 栈与队列 (Stack & Queue)

- **栈 (Stack<T>)**：后进先出（LIFO, Last-In First-Out）。`Push()` 入栈，`Pop()` 出栈，`Peek()` 查看栈顶。应用：函数调用栈、撤销历史（指令模式）、括号匹配。
- **队列 (Queue<T>)**：先进先出（FIFO, First-In First-Out）。`Enqueue()` 入队，`Dequeue()` 出队，`Peek()` 查看队首。应用：BFS 算法、任务队列、消息队列。

4.1.4 链表 (Linked List)

由节点（Node）组成，每个节点包含数据和指向下一个节点的引用。

- **单向链表**：每个节点只有 `next` 指针。插入删除 $O(1)$ （已知位置），访问 $O(n)$ 。
- **双向链表**：`LinkedList<T>`，每个节点有 `Previous` 和 `Next` 指针。支持双向遍历。
- **与数组/List 的对比**：链表插入删除不需移动其他元素，但不支持快速随机访问，且每个节点有额外的指针内存开销。

4.1.5 树 (Tree)

- **二叉树 (Binary Tree)**：每个节点最多两个子节点。
- **二叉搜索树 (BST)**：左子树所有节点 < 根节点 < 右子树所有节点。查找/插入/删除平均 $O(\log n)$ 。
- **堆 (Heap)**：一种特殊的完全二叉树。最小堆的根节点是最小值。常用于优先队列和堆排序。
- **游戏中的应用**：行为树（AI 决策）、场景图（层次变换）、BVH（空间划分与碰撞检测）。

4.2 算法复杂度分析

[p.6]

4.2.1 大 O 表示法 (Big-O Notation)

大 O 表示法描述算法在最坏情况下的时间复杂度，关注输入规模 n 很大时的时间增长趋势。

- $O(1)$ ——**常数时间**：与输入规模无关。如数组索引访问、哈希表查找。

- $O(\log n)$ ——对数时间：每次操作将问题规模减半。如二分查找、平衡 BST 操作。
- $O(n)$ ——线性时间：与输入规模成正比。如遍历数组、线性查找。
- $O(n \log n)$ ——线性对数时间：如归并排序、快速排序（平均）。
- $O(n^2)$ ——平方时间：如冒泡排序、双重循环。
- $O(2^n)$ ——指数时间：如递归计算斐波那契数列（无缓存）。通常意味着算法不可行。

游戏开发中的性能意识：在 `Update()` 中执行的代码每帧都运行，60fps 意味着每秒执行 60 次。因此 `Update()` 中要避免 $O(n^2)$ 操作、频繁的 `FindObjectOfType()`、不必要的 `Instantiate/Destroy` 等。

5 第四部分：设计模式总复习

设计模式（Design Patterns）是面向对象编程中经过验证的、可复用的解决方案。SC102 学习了七种核心设计模式，它们在游戏开发中有直接的应用场景。**注意：**设计模式是思想，不是死板的模板——理解其要解决的问题比记住实现细节更重要。

5.1 单例模式 (Singleton Pattern)

[p.7-8]

问题：如何确保一个类只有一个实例，并提供全局访问？

解决方案：私有构造函数 + 静态实例 + 全局访问属性。

核心代码框架：

```
public class Singleton<T> where T : class, new() {
    protected Singleton() { } // protected, 子类可访问
    private static T _inst = null;
    public static T instance => _inst ?? (_inst = new T());
    public static void Clear() => _inst = null;
}
```

游戏中的应用： `GameManager`、`AudioManager`、`UIManager`、`SceneManager` 等全局管理器。

Unity 中的特殊实现——MonoSingleton：

```
public class MonoSingleton<T> : MonoBehaviour where T : MonoBehaviour {
    private static T _instance;
    public static T Instance {
        get {
            if (_instance == null) {
                _instance = FindObjectOfType<T>();
            }
        }
    }
}
```

```

        if (_instance == null) {
            GameObject go = new GameObject(typeof(T).Name);
            _instance = go.AddComponent<T>();
        }
    }
    return _instance;
}
}

protected virtual void Awake() {
    if (_instance == null) {
        _instance = this as T;
        DontDestroyOnLoad(gameObject); // 跨场景保持
    } else if (_instance != this) {
        Destroy(gameObject);           // 销毁重复实例
    }
}
}
}

```

使用警示：单例容易导致过度耦合和难以测试的代码。在大型项目中，优先考虑依赖注入（Dependency Injection）、ScriptableObject 或 static 工具类作为替代方案。

5.2 指令模式 (Command Pattern)

[p.7,9]

问题：如何实现灵活的撤销/重做功能？如何解耦操作的”发出者”和”执行者”？

解决方案：将每个操作封装为独立的指令对象，每个对象实现了 `Execute()` 和 `Undo()` 方法。

五大组成部分：

1. **ICommand (指令接口)**：定义 `Execute/Undo` 签名
2. **ConcreteCommand (具体指令类)**：实现具体操作逻辑
3. **Invoker (调用者)**：管理和执行指令，维护历史栈
4. **Receiver (接收者)**：真正执行操作的底层对象
5. **Client (客户端)**：创建指令并交给 `Invoker`

核心实现：

```

public class CommandInvoker {
    private Stack<ICommand> commandHistory = new Stack<ICommand>();

    public void ExecuteCommand(ICommand command) {

```



```

        command.Execute();
        commandHistory.Push(command);
    }

    public void Undo() {
        if (commandHistory.Count > 0)
            commandHistory.Pop().Undo();
    }
}

```

游戏中的应用：关卡编辑器中的每一步编辑（Move、Place、Delete）、回合制游戏的行动序列、战斗重播系统、宏录制与回放。

5.3 工厂模式 (Factory Pattern)

[p.7,10]

问题：如何将对象的创建逻辑集中管理，并隐藏具体类型的实例化细节？

简单工厂 (Simple Factory)：

将创建逻辑封装在一个工厂方法中，调用者不需要知道具体创建了哪个子类对象。

```

public static class AnimalFactory {
    public static Animal CreateAnimal(string type) {
        return type switch {
            "Dog" => new Dog(),
            "Cat" => new Cat(),
            _ => throw new ArgumentException($"Unknown type: {type}")
        };
    }
}

```

抽象工厂 (Abstract Factory)：

定义一个工厂接口，允许多个工厂实现创建不同系列（产品族）的对象。

两种工厂的核心区别：

- **简单工厂：**生产单一产品体系（如 Animal），通过参数选择产品变体。
- **抽象工厂：**生产产品族（如红军工厂生产 RedDog+RedCat，蓝军工厂生产 BlueDog+BlueCat），运行时切换整个工厂即可切换整个系列。

游戏中的应用：敌人生成系统（不同关卡的敌人类型不同）、UI 主题工厂（不同风格的按钮/面板）、技能效果工厂。

5.4 观察者模式 (Observer Pattern)

[p.7,10]

问题：当一个对象的状态变化时，如何自动通知所有依赖它的对象？

解决方案：Subject（主题/通知者）维护一个 Observer（观察者）列表，状态变化时遍历列表通知所有 Observer。

C# 内置实现：C# 的 event Action / EventHandler 本质上就是观察者模式的语言级实现。Unity 的 UnityEvent（如 Button.onClick）也是观察者模式。

手动实现：

```
public interface IObservable { void OnNotify(string eventName, object data); }

public class Subject {
    private List<IObservable> observers = new List<IObservable>();

    public void Register(IObservable obs) { observers.Add(obs); }
    public void Unregister(IObservable obs) { observers.Remove(obs); }
    public void Notify(string eventName, object data = null) {
        for (int i = observers.Count - 1; i >= 0; i--) // 逆向遍历!
            observers[i].OnNotify(eventName, data);
    }
}
```

注意：逆向遍历（从后往前）是为了防止在通知过程中某个 Observer 被移除导致的索引错乱。

游戏中的应用：成就系统（监听击杀数）、UI 更新（监听血量变化）、任务系统（监听任务完成事件）、全局事件总线（Event Bus）。

5.5 策略模式 (Strategy Pattern)

[p.7,11]

问题：如何在运行时根据需要灵活切换算法/行为？

解决方案：将每种算法封装成独立的策略类，持有者通过替换策略对象来改变行为。

```
public interface IDamageStrategy {
    void TakeDamage(float damage);
}

public class WaterDamageStrategy : IDamageStrategy { ... }
public class LandDamageStrategy : IDamageStrategy { ... }

public class Player {
    private IDamageStrategy damageStrategy;
```

```

public void SetStrategy(IDamageStrategy s) => damageStrategy = s;
public void TakeDamage(float d) => damageStrategy.TakeDamage(d);
}

```

核心价值：消除冗长的 if-else 或 switch 链；遵循开闭原则（新增策略不需要修改已有代码）；策略可独立测试。

游戏中的应用：AI 行为切换（巡逻、追击、逃跑）、攻击方式切换、移动方式切换、难度自适应策略。

5.6 对象池 (Object Pool)

[p.7,11]

问题：在 Unity 中，频繁的 Instantiate() 和 Destroy() 操作会导致严重的性能问题（GC 分配、内存碎片）。

解决方案：预先创建一批对象，用 SetActive() 代替 Destroy，用 SetActive(true) 代替 Instantiate。

基本实现：

```

public class ObjectPool : MonoBehaviour {
    public GameObject objectToPool;
    public int initialPoolSize = 10;
    private List<GameObject> pool;

    void Start() {
        pool = new List<GameObject>();
        for (int i = 0; i < initialPoolSize; i++) {
            GameObject obj = Instantiate(objectToPool);
            obj.SetActive(false);
            pool.Add(obj);
        }
    }

    public GameObject Get() {
        foreach (var obj in pool)
            if (!obj.activeInHierarchy) { obj.SetActive(true); return obj; }
        // 池中无空闲则扩容
        GameObject newObj = Instantiate(objectToPool);
        pool.Add(newObj);
        return newObj;
    }

    public void ReturnToPool(GameObject obj) {

```

```

        obj.SetActive(false);
    }
}

```

高级对象池特性：

- **多类型管理：**使用 `Dictionary<Type, Queue<GameObject>>` 管理不同类型对象的池。
- **动态扩缩容：**根据使用率自动调整池大小，避免内存浪费。
- **对象状态重置：**回收时重置位置、血量、动画状态等，确保取出的是”干净”对象。
- **泛型化：**支持任意 `Component` 类型，如 `ObjectPool<T> where T : Component`。

适用场景：子弹 (Bullet)、粒子特效 (Particle Effect)、敌人、掉落物品、UI 弹窗提示等高频创建/销毁的对象。

5.7 ECS 模式 (Entity-Component-System)

[p.7,12-13]

问题：传统的 `GameObject-Component` 模式在拥有大量实体时面临性能瓶颈（分散的内存布局、缓存不友好、难以并行化）。

解决方案：将数据 (Component) 与逻辑 (System) 彻底分离，实体 (Entity) 仅为 ID 标识。

5.7.1 ECS 三大核心

- **Entity (实体)：**一个整数 ID，代表游戏世界中的对象。实体本身不包含任何数据或行为。
- **Component (组件)：**纯数据结构 (Plain Data)，只包含字段，不包含任何方法。例如 `Position` 只存储 `x`、`y`、`z` 坐标；`Velocity` 只存储速度分量。
- **System (系统)：**纯逻辑，遍历所有拥有特定 `Component` 组合的 `Entity` 并执行操作。例如 `MovementSystem` 找到所有同时有 `Position` 和 `Velocity` 组件的 `Entity`，逐帧更新位置。

5.7.2 ECS vs 传统 OOP 的本质区别

维度	传统 OOP (Unity 默认)	ECS
数据结构	数据和方法混合在 <code>Component</code> 里	<code>Component</code> 只存纯数据
逻辑位置	在每个 <code>MonoBehaviour</code> 的 <code>Update</code> 中	集中在 <code>System</code> 中批量处理
内存布局	对象分散在堆内存各处	同类型 <code>Component</code> 连续存储 (Cache-Friendly)
并行性	难以安全多线程	<code>System</code> 天然可并行

5.7.3 ECS 的性能优势

1. **Cache-Friendly (缓存友好)**: 相同类型的 Component 在内存中连续排列, CPU 缓存 (Cache Line) 可以一次性加载多个组件数据, 大幅减少缓存未命中 (Cache Miss)。
2. **按需组合 (Lean Entities)**: 实体只拥有它需要的组件, 无冗余数据。
3. **天然并行 (Parallelizable)**: System 只操作 Component 数组, 各实体间无直接引用依赖, 可安全地启用多线程 (如 C# Job System)。

5.7.4 Unity DOTS 简介

[p.13]

Unity 的 ECS 实现称为 **DOTS (Data-Oriented Technology Stack)**, 包含三个核心组件:

- **Entities 包**: ECS 的核心实现, 提供 Entity、ComponentData、SystemBase 等。
- **C# Job System**: 安全的多线程框架, 编译器静态验证数据竞争, 避免死锁和竞态条件。
- **Burst Compiler**: 基于 LLVM 的 AOT 编译器, 将 C# 子集编译为高度优化的原生代码 (SIMD 指令等), 性能提升可达数倍到数十倍。

DOTS 的适用场景: 同屏大量实体 (RTS 的单位、大世界的植被、弹幕游戏的子弹)、大规模物理模拟、粒子系统、群集 AI 等。

6 第五部分: 图形学与 Shader 基础回顾

6.1 渲染管线 (Rendering Pipeline)

[p.14,16]

渲染管线 (Rendering Pipeline / Graphics Pipeline) 描述了从 3D 场景数据到最终 2D 屏幕图像的完整处理流程。理解渲染管线是学习 Shader 编程的前提。

6.1.1 渲染管线的四大阶段

[p.16]

1. 应用程序阶段 (Application Stage):

- 在 CPU 上执行, 完全由开发者控制。
- **准备渲染数据**: 视锥体剔除 (Frustum Culling)、LOD 选择、设置渲染状态 (材质、Shader 参数)。
- **发起 Draw Call**——将渲染指令提交给 GPU。

2. 几何处理阶段 (Geometry Processing Stage):

- 在 GPU 上执行，主要处理顶点数据。
- 包含：顶点着色器 (Vertex Shader) —— 对每个顶点执行坐标变换 (模型空间 → 世界空间 → 观察空间 → 裁剪空间)，计算顶级别的光照数据。
- 可选阶段：曲面细分着色器 (Tessellation Shader，动态生成更多三角形)、几何着色器 (Geometry Shader，可增删顶点)。
- 裁剪 (Clipping) —— 剔除视野外的图元。屏幕映射 (Screen Mapping) —— 将 NDC 坐标映射到屏幕像素坐标。

3. 光栅化阶段 (Rasterization Stage):

- 将三角形 (或线段、点) 转换为屏幕上的像素片段 (Fragment)。
- 对三角形内部的每个像素，通过插值 (Interpolation) 计算其属性 (颜色、UV 坐标、法线等)。

4. 像素处理阶段 (Pixel Processing Stage):

- 片元着色器 (Fragment Shader / Pixel Shader): 对每个片元计算最终颜色。执行纹理采样、光照计算、阴影等。
- 逐片元操作 (Per-Fragment Operations): 深度测试 (Depth Test / Z-Test)、模板测试 (Stencil Test)、透明度混合 (Alpha Blending)。
- 最终将颜色写入帧缓冲 (Frame Buffer)，显示到屏幕。

6.1.2 坐标空间变换 (Coordinate Space Transformations)

[p.14]

3D 渲染过程中，顶点坐标经历多次空间变换：

- 模型空间 (Model Space / Local Space): 以模型自身为原点的坐标。建模时的原始坐标。
- 世界空间 (World Space): 所有物体共享的全局坐标。通过 Model Matrix 将模型空间变换到世界空间。
- 观察空间 (View Space / Camera Space): 以摄像机为原点的坐标。通过 View Matrix 变换，摄像机朝向始终为-z 轴。
- 裁剪空间 (Clip Space): 通过 Projection Matrix (透视或正交) 变换后的坐标。超出 $[-w, w]$ 范围的顶点被裁剪。
- 屏幕空间 (Screen Space): 最终显示在屏幕上的像素坐标。通过视口变换 (Viewport Transform) 从 NDC 映射得到。

核心变换链: $V_{clip} = M_{projection} \cdot M_{view} \cdot M_{model} \cdot V_{local}$

6.2 Shader 编程基础

[p.14-15]

6.2.1 什么是 Shader?

Shader（着色器）是运行在 GPU 上的小程序，控制渲染管线的特定阶段（最核心的是顶点着色器和片元着色器）。每个 Shader 定义了物体的视觉表现——包括颜色、纹理、光照、透明度、特殊效果等。

6.2.2 Unity 中的 Shader 类型

- **表面着色器 (Surface Shader)**: Unity 特有的高级 Shader 抽象。开发者编写表面属性（反照率、法线、光滑度等），Unity 自动生成完整的顶点/片元着色器。适合大部分常见材质需求，开发效率高。
- **顶点/片元着色器 (Vertex/Fragment Shader)**: 完全手动控制渲染管线的顶点和片元阶段。更灵活，但也更复杂。适合需要非标准光照模型或特殊效果的情况。
- **固定函数着色器 (Fixed Function Shader)**: 已被淘汰的旧式 Shader，仅用于极老的硬件兼容。

6.2.3 ShaderLab 语法简介

[p.15]

Unity 的 Shader 使用 ShaderLab 语法编写，是一种结合了声明式属性和 Cg/HLSL 代码的混合语言。

```
Shader "Custom/MyShader" {
    Properties {
        _MainTex ("Main Texture", 2D) = "white" {}
        _Color ("Color", Color) = (1, 1, 1, 1)
    }
    SubShader {
        Tags { "RenderType"="Opaque" }
        Pass {
            CGPROGRAM
            #pragma vertex vert
            #pragma fragment frag
            // HLSL/Cg 代码 ...
            ENDCG
        }
    }
}
```

```
}
```

6.2.4 光照模型 (Lighting Models)

[p.14]

- **漫反射 (Diffuse Reflection / Lambert):** 光线均匀散射到各个方向, 与观察角度无关。
 $C_{diffuse} = C_{light} \cdot C_{material} \cdot \max(0, N \cdot L)$, 其中 N 为表面法线, L 为光线方向。
- **高光反射 (Specular Reflection):** 光线在特定方向集中反射, 产生”亮点”。常见模型有 Phong 和 Blinn-Phong。
- **环境光 (Ambient Light):** 模拟间接光照的近似值, 通常为常数低亮度颜色, 防止完全无光区域变成纯黑。
- **Fresnel 效应:** 掠射角处的反射更强 (如看水面远处时反射更明显)。

7 第六部分: 游戏引擎架构概览

[17] 理解游戏引擎的宏观架构有助于在开发中做出更好的设计决策, 也为后续引擎工具开发课程做准备。

7.1 引擎核心系统

- **内存管理 (Memory Management):** 自定义内存分配器 (池分配器、栈分配器)、内存碎片管理、引用计数与 GC 策略。
- **文件系统 (File I/O):** 资源的读取、序列化/反序列化、打包 (AssetBundle)、异步加载。
- **日志系统 (Logging):** 分级日志 (Debug/Info/Warning/Error)、性能开销控制、发布版本的日志裁剪。

7.2 各子引擎

- **渲染引擎 (Rendering Engine):** 管理所有可见物体的绘制。决定渲染顺序 (不透明前向/后向、透明后向)、批处理 (Static/Dynamic Batching、GPU Instancing)、LOD 系统。
- **物理引擎 (Physics Engine):** 碰撞检测 (Broad Phase / Narrow Phase)、刚体模拟、关节约束、射线检测。Unity 内置 NVIDIA PhysX (3D) 和 Box2D (2D)。
- **音频系统 (Audio System):** 音效播放、3D 空间音频、音频混合、动态音乐系统。
- **脚本系统 (Scripting System):** 脚本语言集成 (C# 通过 Mono/IL2CPP)、脚本生命周期管理、脚本间通信。

- **UI 系统 (UI System):** UGUI/UIToolkit、Canvas 渲染模式 (Overlay/Camera/WorldSpace)、布局系统、事件系统。
- **网络系统 (Networking System):** 客户端-服务器模型、P2P、状态同步、RPC、预测与插值。
- **动画系统 (Animation System):** Animator 状态机、AnimationClip、骨骼动画、IK (Inverse Kinematics)、动画混合与遮罩。

7.3 游戏循环 (Game Loop)

游戏引擎的核心是**游戏循环 (Game Loop)**，每帧执行以下步骤：

1. 处理输入 (Input)
2. 更新游戏逻辑 (Update)
3. 物理模拟 (FixedUpdate)
4. 渲染 (Render)
5. 等待下一帧 (VSync)

在 Unity 中，这个循环被封装在引擎内部，开发者通过在 MonoBehaviour 的对应回调方法中编写逻辑来参与游戏的每一帧。

8 第七部分：未来展望

这是 SC102 课程的展望部分，为有兴趣深入的同学提供进阶学习方向指导。以下内容不作为期末考试范围，但对后续课程和实际游戏开发有重要参考价值。

8.1 方向一：Shader 与渲染进阶

[p.18]

8.1.1 高级着色语言 (HLSL/GLSL)

在表面着色器的基础上深入学习 HLSL (High-Level Shading Language, DirectX) 或 GLSL (OpenGL Shading Language)，能够编写完全自定义的顶点/片元着色器：

- 手动控制顶点变换、法线处理
- 自定义光照模型 (卡通渲染/Toon Shading、各向异性高光)
- 程序化纹理生成 (Procedural Texture)
- 顶点动画 (如草地摆动、布料模拟)

8.1.2 PBR (Physically Based Rendering)

基于物理的渲染（PBR）是目前 3A 游戏的标准渲染方案。其核心思想是使用物理上正确的光照和材质模型，使物体在不同光照环境下都看起来真实。

- **核心材质参数：**反照率（Albedo）、金属度（Metallic）、光滑度（Smoothness）、法线（Normal）、环境光遮蔽（Ambient Occlusion）。
- **核心概念：**能量守恒（反射光能量不超过入射光）、微表面模型（Microfacet Model）、BRDF（双向反射分布函数）。
- **Unity 中的 PBR：**Standard Shader（内置）和 Lit Shader（URP/HDRP）都是 PBR 实现。

8.1.3 后处理效果 (Post-Processing)

后处理是在渲染完成后对最终图像施加的屏幕空间效果：

- **Bloom（泛光）：**亮区溢出到周围像素
- **Color Grading（颜色分级）：**调整画面的色调/饱和度/对比度
- **Depth of Field（景深）：**模拟相机焦外模糊
- **Motion Blur（运动模糊）：**快速移动物体产生拖影
- **Ambient Occlusion（环境光遮蔽）：**增强局部阴影细节
- **Anti-Aliasing（抗锯齿）：**FXAA、SMAA、TAA 等

8.1.4 计算着色器 (Compute Shader)

与图形着色器不同，计算着色器不参与渲染管线，而是利用 GPU 的大规模并行计算能力处理通用任务：

- GPU 粒子系统的物理模拟
- 大规模矩阵运算
- 程序化地形生成
- 布料/流体模拟

8.1.5 实时光线追踪 (Real-time Ray Tracing)

光线追踪通过模拟光线的物理传播路径，能够实现极其逼真的反射、折射、全局光照和软阴影效果。NVIDIA RTX 系列显卡和 DirectX Raytracing（DXR）API 已使实时光追成为可能。Unity HDRP 支持实时光线追踪。

8.1.6 SRP (Scriptable Render Pipeline)

Unity 的可编程渲染管线 (SRP) 允许开发者自定义整个渲染流程：

- **URP (Universal Render Pipeline)**: 面向多平台的高性能渲染方案
- **HDRP (High Definition Render Pipeline)**: 面向高端平台的电影级渲染方案
- **自定义 RenderPipeline**: 从零设计渲染流程，完全控制 Culling、Filtering、Rendering Pass 等

8.2 方向二：引擎工具开发

[p.19]

8.2.1 编辑器扩展 (Editor Scripting)

为 Unity 编辑器编写自定义工具，提升团队开发效率：

- **Custom Inspector**: 自定义组件在 Inspector 面板的显示
- **Editor Window**: 创建独立的编辑器窗口（如关卡编辑器、数据可视化工具）
- **Property Drawer**: 自定义特定类型字段的绘制方式
- **Gizmos**: 在 Scene 视图中绘制辅助图形（如 AI 路径、触发器范围）
- **MenuItem**: 添加自定义的编辑器菜单项

8.2.2 资源导入管线 (Asset Pipeline)

- **AssetPostprocessor**: 在资源导入时执行自定义处理（如自动设置纹理格式、生成 Mipmap）。
- **ScriptedImporter**: 自定义导入器，支持非标准资源格式。
- **AssetBundle**: 资源打包与动态加载策略。
- **Addressables**: Unity 的新一代资源管理系统，比 AssetBundle 更灵活易用。

8.2.3 程序化生成 (Procedural Generation)

程序化内容生成 (PCG) 通过算法自动创建游戏内容，减少手工工作量：

- **地形生成**: Perlin 噪声、分形布朗运动 (FBM)、侵蚀模拟
- **关卡生成**: 房间布局算法 (BSP、细胞自动机)、Roguelike 地图
- **建筑/道路**: L-system、形状语法

8.2.4 性能分析与优化 (Profiling & Optimization)

- **Unity Profiler:** 分析 CPU 耗时、GPU 耗时、内存分配、渲染统计 (Draw Call、Batches、SetPass calls)
- **Frame Debugger:** 逐帧逐 Draw Call 查看渲染过程
- **Memory Profiler:** 分析内存使用、查找内存泄漏
- **常见优化策略:** 批处理 (Batching)、LOD (Level of Detail)、遮挡剔除 (Occlusion Culling)、对象池、纹理压缩、网格合并

8.2.5 CI/CD 自动化构建

- 使用 Unity 命令行模式实现自动化构建
- 与 Jenkins、GitHub Actions 等 CI 工具集成
- 自动测试、自动打包、自动上传分发

8.3 方向三：进阶游戏开发技术

[p.20]

8.3.1 网络同步 (Networking & Sync)

- **权威服务器模型 (Authoritative Server):** 服务器为最终权威，客户端为“哑终端”。
- **状态同步 vs 帧同步:** 同步状态数据 vs 同步玩家输入指令。
- **预测与回滚:** 客户端预测 (Client-Side Prediction) + 服务器回滚 (Server Reconciliation) 减少延迟感。
- **插值 (Interpolation):** 在两个收到的网络状态之间平滑过渡。

8.3.2 AI 系统

- **有限状态机 (Finite State Machine, FSM):** 最基本的 AI 结构，状态 + 转换条件。简单清晰但状态多时难以维护。
- **行为树 (Behavior Tree):** 层级化的 AI 决策结构，由 Selector (或节点)、Sequence (与节点)、Decorator (修饰节点) 等组成。比 FSM 更模块化、更易扩展。
- **导航网格 (NavMesh):** Unity 内置的寻路系统，自动生成可行走区域，支持动态障碍物。
- **进阶方向:** 效用 AI (Utility AI)、目标导向行动规划 (GOAP)、机器学习 (ML-Agents)。

8.3.3 物理模拟进阶

- 自定义物理材质的弹力/摩擦力
- 铰链关节（Hinge Joint）、弹簧关节（Spring Joint）、可配置关节（Configurable Joint）
- 布娃娃系统（Ragdoll）
- 载具物理（Wheel Collider）

8.4 方向四：计算机图形学进阶

[p.21]

- **实时全局光照 (Real-time GI)**: 使用光照探针 (Light Probe)、反射探针 (Reflection Probe)、Enlighten/Progressive Lightmapper 等技术模拟间接光照。
- **体积渲染 (Volumetric Rendering)**: 体积光 (God Rays)、体积雾 (Volumetric Fog)、云层渲染。
- **GPU 粒子系统**: 使用 Compute Shader 在 GPU 上模拟数百万粒子 (Unity VFX Graph)。
- **地形渲染**: 基于高度的地形系统、纹理混合 (Splat Map)、LOD 分块 (Terrain Chunk LOD)。
- **动画系统进阶**: 动画融合树 (Blend Tree)、动画层 (Animation Layer)、逆向运动学 (IK)、程序化动画 (如根据地形自适应脚部位置)。

9 第八部分：学习路径与总结

9.1 SC102 完整学习路径

[p.22]

第一阶段：语言与工具基础 C# 语法基础 → Unity 界面与操作 → MonoBehaviour 编程
第二阶段：软件工程思维 面向对象编程 (OOP) → SOLID 原则 → 数据结构与算法 → 设计模式
第三阶段：引擎与图形学 Unity 进阶 → 渲染管线理解 → Shader 编程入门 → 引擎架构认知
第四阶段：进阶专精（未来方向） Shader 渲染 引擎工具开发 网络同步 AI 系统 图形学进阶

9.2 课程总结

[p.23]

SC102 完成了从编程基础到游戏开发的完整入门教育。通过本课程，学生应当：

- 能够使用 C# 编写结构清晰、可维护的游戏代码
- 理解面向对象编程的核心理念和设计模式的价值
- 掌握基本的数据结构及其性能特征
- 了解 3D 渲染管线和 Shader 编程的基本概念
- 对游戏引擎的宏观架构有基本认知
- 能够独立完成中小规模的游戏项目

9.3 后续课程推荐

[p.23,25]

- **SC103/SC104: 进阶图形学与 Shader 编程**——深入渲染管线、自定义 Shader 开发、视觉效果实现
- **游戏引擎架构**：深入引擎底层，了解内存管理、资源系统、多线程架构等
- **引擎工具开发**：编辑器扩展、自动化流程、团队协作工具链
- **独立项目实践**：综合运用 SC102 所有知识，从设计到发布完成一个完整游戏

9.4 寄语

[p.24]

编程是手艺，需要不断练习。

游戏开发是综合学科，编程、美术、设计、音效各有深度。

保持好奇心，多写代码，多看别人怎么写，多思考为什么这么设计。

SC102 只是开始，精彩的世界才刚刚展开。

10 期末重点知识自检清单

以下为 SC102 期末可能涉及的核心考点，建议逐条确认理解到位：

10.1 C# 基础

- 值类型与引用类型的区别及内存模型
- ref/out 关键字的使用场景
- 方法重载的条件（参数列表不同）
- Lambda 表达式与委托（delegate/Action/Func）的基本使用

10.2 面向对象编程

- 封装、继承、多态、抽象的定义与 C# 实现方式
- virtual/override/new 关键字的含义和区别
- 抽象类（abstract）与接口（interface）的区别
- SOLID 原则的基本含义

10.3 数据结构

- List vs Array 的底层原理和性能差异
- Dictionary 的查找原理（哈希表）和键的要求
- Stack（LIFO）和 Queue（FIFO）的特征和应用场景
- 常用操作的时间复杂度判断（索引、添加、删除、查找）

10.4 设计模式（重点）

- 七种设计模式各自解决的核心问题是什么？
- 单例模式：如何保证唯一性？普通单例 vs MonoSingleton 的区别？
- 指令模式：五个组成部分的关系？为什么用 Stack 存储历史？
- 工厂模式：简单工厂 vs 抽象工厂的使用场景？
- 观察者模式：如何手动实现？为何逆向遍历 Observer 列表？
- 策略模式：如何消除 if-else 链？
- 对象池：为什么 Instantiate/Destroy 性能差？对象池如何解决？
- ECS：与 Unity 传统 GameObject 体系的核心区别？为什么 ECS 性能更好？

10.5 图形学基础

- 渲染管线的四个阶段及其主要功能
- 顶点着色器 vs 片元着色器的职责区别
- 坐标空间变换链 (Model $\rightarrow$ World $\rightarrow$ View $\rightarrow$ Clip)
- 漫反射光照公式的基本理解: $N \cdot L$ 的含义

11 复习方法建议

1. **代码复盘**: 将本学期写过的关键代码重写一遍 (单例基类、对象池、指令管理等)。
2. **概念对比**: 制作对比表 (如 OOP 四大特性、设计模式要点、数据结构复杂度)。
3. **动手实现**: 不看参考资料, 从零实现一种设计模式或数据结构。
4. **查漏补缺**: 对照上述自检清单, 标记不熟悉的知识点重点复习。
5. **场景联想**: 为每个知识点想一个游戏开发中的实际应用场景 (如 “什么时候用 Stack?” “撤销功能!”)。